

GADGETXPRESS

Sistema de Gestión de Inventario

Reporte de Refactorización · Evidencia 3

Universidad Pascual Bravo



Miguel Angel Cardona

Tecnología en Desarrollo de Software

Universidad Pascual Bravo

logica de programacion

Principio de Responsabilidad Única

Evidencia 3 · Programación · 2025

Proyecto:	Gestión de Inventario en C++
Versión:	2.0 — Código Refactorizado
Lenguaje:	C++
Enfoque:	Refactorización — Principio de Responsabilidad Única

1. Introducción y Contexto

GadgetXpress necesitaba pasar de un código que «simplemente funcionaba» a un software de calidad profesional: mantenible, escalable y fácil de entender. Esta evidencia aplica el principio de responsabilidad única (SRP) para lograr ese objetivo.

El código original tenía toda la lógica concentrada en pocas funciones y en la función main. Si el equipo quería agregar nuevas funcionalidades (como simulación de ventas o exportación de reportes), tendría que modificar bloques grandes de código mezclado, lo cual aumenta el riesgo de introducir errores y dificulta las pruebas.

La refactorización no cambia lo que hace el programa, sino cómo está organizado internamente.

2. Análisis del Código Original

Antes de refactorizar, se identificaron los siguientes problemas en el código original:

- La función main contenía toda la lógica de registro inicial (un bucle for con cin para 3 productos hardcodeados).
- No existía función para agregar productos dinámicamente ni para listarlos.
- La lógica de búsqueda por código estaba duplicada implícitamente (en actualizarCantidad y potencialmente en futuros módulos).
- Se usaban arreglos de tamaño fijo (Producto inventario[capacidad]) en vez de estructuras de datos dinámicas.
- No había validación de entradas del usuario, lo que podría crashear el programa.
- Los caracteres con tilde mostraban encoding corrupto (tamaÃ±o en lugar de tamaño).

3. Estrategia de Refactorización

La refactorización se dividió en cuatro pasos, siguiendo exactamente las instrucciones de la tarea:

3.1 Paso 1 — Funciones de Responsabilidad Única

Se identificaron todas las tareas discretas del programa y se encapsuló cada una en su propia función. La siguiente tabla muestra la correspondencia:

Responsabilidad	Función creada	Descripción breve
Mostrar menú	mostrarMenu()	Imprime las 5 opciones del menú principal
Agregar producto	agregarProducto(...)	Pide datos al usuario y registra el nuevo producto
Listar inventario	listarProductos(...)	Muestra todos los productos con formato tabular
Actualizar cantidad	actualizarCantidad(...)	Busca un producto y modifica su cantidad
Reporte bajo inventario	generarReporteBajoInventario(...)	Filtra y muestra productos con menos de 5 unidades

3.2 Paso 2 — Funciones de Ayuda Reutilizables

Se extrajeron tres funciones de ayuda para eliminar la duplicidad:

Función de ayuda	¿Por qué se creó?
buscarProducto(...)	Centraliza la búsqueda por código. Se usa tanto en agregarProducto (validar duplicados) como en actualizarCantidad. Sin ella, esa lógica estaría copiada en dos lugares.
leerEnteroPositivo(...)	Valida que el usuario ingrese un entero ≥ 0 . Evita duplicar el bucle de validación en agregarProducto y actualizarCantidad.
leerDecimalPositivo(...)	Igual que la anterior, pero para valores decimales (precios). Garantiza consistencia en la validación.
limpiarBuffer()	Limpia cin después de errores. Centraliza esta tarea técnica en un solo lugar para no repetirla en múltiples funciones.

3.3 Paso 3 — Simplificación del main

El programa principal se convirtió en un controlador de alto nivel: su única responsabilidad es ejecutar el bucle principal y despachar las opciones a las funciones correspondientes mediante switch.

4. Antes y Después de la función main

4.1 Función main — ANTES

El main original tenía 3 responsabilidades mezcladas: registro inicial de datos, control del menú y la lógica de impresión del menú embebida.

```
int main() {
    int capacidad = 3;
    Producto inventario[capacidad];
    cout << "SISTEMA GADGETXPRESS - REGISTRO INICIAL" << endl;
    for (int i = 0; i < capacidad; i++) {
        cout << "Nombre: "; cin >> inventario[i].nombre;
        cout << "Codigo: "; cin >> inventario[i].codigo;
        cout << "Cantidad: "; cin >> inventario[i].cantidad;
        cout << "Precio: "; cin >> inventario[i].precio;
    }
    int opcion;
    do {
        cout << "--- MENU PRINCIPAL ---" << endl;
        cout << "1. Actualizar cantidad" << endl;
        cout << "2. Generar reporte" << endl;
        cout << "3. Salir" << endl;
        cin >> opcion;
        switch (opcion) {
            case 1: actualizarCantidad(inventario, capacidad); break;
            case 2: generarReporte(inventario, capacidad); break;
            case 3: cout << "Saliendo..." << endl; break;
            default: cout << "Opcion no valida." << endl;
        }
    } while (opcion != 3);
    return 0;
}
```

4.2 Función main — DESPUÉS

El main refactorizado es un controlador puro: solo inicializa el vector, llama a mostrarMenu() y despacha con switch. Cero lógica de negocio.

```
int main() {
    vector<Producto> inventario;
    int opcion;
    do {
        mostrarMenu();
        cin >> opcion;
        switch (opcion) {
            case 1: agregarProducto(inventario); break;
            case 2: listarProductos(inventario); break;
            case 3: actualizarCantidad(inventario); break;
            case 4: generarReporteBajoInventario(inventario); break;
            case 5: cout << "Hasta luego!" << endl; break;
            default: cout << "Opcion no valida." << endl;
        }
    } while (opcion != 5);
    return 0;
}
```

5. Resumen de Mejoras Introducidas

Aspecto	Versión Original	Versión Refactorizada
Inventario	Array fijo de 3 productos	vector<Producto> dinámico (sin límite)
Agregar producto	Solo en el main, hardcodeado	Función agregarProducto() con validación de duplicados
Listar productos	No existía	Función listarProductos() con formato tabular alineado
Búsqueda por código	Código incrustado en actualizarCantidad	Función buscarProducto() reutilizable en todo el programa
Validación de entrada	No existía	leerEnteroPositivo() y leerDecimalPositivo() con bucle de reintento
Función main	~30 líneas con lógica mixta	~15 líneas, solo es un controlador de flujo
Opciones del menú	3 opciones	5 opciones (se agregó Agregar y Listar)

6. Beneficios de la Nueva Estructura

1. **Mantenibilidad:** Si en el futuro se necesita cambiar cómo se valida el código de un producto, solo se modifica la función buscarProducto(). No hay que rastrear el mismo código en varios lugares.
2. **Escalabilidad:** Agregar una nueva funcionalidad (por ejemplo, simulación de ventas) consiste en crear una nueva función y añadir un case al switch. El resto del código no se toca.
3. **Legibilidad:** Cada función tiene un nombre que describe exactamente qué hace. Cualquier desarrollador que lea el código entiende la estructura sin necesidad de seguir línea por línea.
4. **Reutilización:** Las funciones de ayuda (buscarProducto, leerEnteroPositivo, etc.) se pueden llamar desde cualquier parte del programa sin duplicar código.
5. **Testabilidad:** Funciones pequeñas con una sola responsabilidad son más fáciles de probar de forma aislada, lo que reduce el riesgo de bugs al hacer cambios.

7. Conclusión

La refactorización transformó un programa funcional pero difícil de mantener en una aplicación modular, limpia y lista para crecer. El código nuevo tiene el mismo comportamiento externo que el original, pero su estructura interna es completamente diferente: cada función hace una sola cosa, no hay lógica duplicada y el main actúa únicamente como controlador de alto nivel.

Esta base sólida permite al equipo de GadgetXpress agregar nuevas funcionalidades de forma confiable y sin miedo a romper lo que ya funciona.